

```
In [1]: import pandas as pd
```

```
In [2]: import numpy as np
import matplotlib.pyplot as plt
```

```
In [3]: df = pd.read_csv("Cars.csv")
```

```
In [4]: df.head()
```

Out[4]:

	name	year	selling_price	km_driven	fuel	seller_type	transmission	owner	mileage	engine	max_power	torque
0	Maruti Swift Dzire VDI	2014	450000	145500	Diesel	Individual	Manual	First Owner	23.4 kmpl	1248 CC	74 bhp	190Nm@ 2000rpm
1	Skoda Rapid 1.5 TDI Ambition	2014	370000	120000	Diesel	Individual	Manual	Second Owner	21.14 kmpl	1498 CC	103.52 bhp	250Nm@ 1500-2500rpm
2	Honda City 2017-2020 EXi	2006	158000	140000	Petrol	Individual	Manual	Third Owner	17.7 kmpl	1497 CC	78 bhp	12.7@ 2,700(kgm@ rpm)
3	Hyundai i20 Sportz Diesel	2010	225000	127000	Diesel	Individual	Manual	First Owner	23.0 kmpl	1396 CC	90 bhp	22.4 kgm at 1750-2750rpm
4	Maruti Swift VXi BSIII	2007	130000	120000	Petrol	Individual	Manual	First Owner	16.1 kmpl	1298 CC	88.2 bhp	11.5@ 4,500(kgm@ rpm)

```
In [5]: df.tail()
```

Out[5]:

	name	year	selling_price	km_driven	fuel	seller_type	transmission	owner	mileage	engine	max_power	torq
8123	Hyundai i20 Magna	2013	320000	110000	Petrol	Individual	Manual	First Owner	18.5 kmpl	1197 CC	82.85 bhp	113.7Nm@ 4000rpm
8124	Hyundai Verna CRDi SX	2007	135000	119000	Diesel	Individual	Manual	Fourth & Above Owner	16.8 kmpl	1493 CC	110 bhp	24@ 1,900-2,750(kgm@ rpm)
8125	Maruti Swift Dzire ZDi	2009	382000	120000	Diesel	Individual	Manual	First Owner	19.3 kmpl	1248 CC	73.9 bhp	190Nm@ 2000rpm
8126	Tata Indigo CR4	2013	290000	25000	Diesel	Individual	Manual	First Owner	23.57 kmpl	1396 CC	70 bhp	140Nm@ 1800-3000rpm
8127	Tata Indigo CR4	2013	290000	25000	Diesel	Individual	Manual	First Owner	23.57 kmpl	1396 CC	70 bhp	140Nm@ 1800-3000rpm

```
In [6]: df.describe()
```

Out[6]:

	year	selling_price	km_driven	seats
count	8128.000000	8.128000e+03	8.128000e+03	7907.000000
mean	2013.804011	6.382718e+05	6.981951e+04	5.416719
std	4.044249	8.062534e+05	5.655055e+04	0.959588
min	1983.000000	2.999900e+04	1.000000e+00	2.000000
25%	2011.000000	2.549990e+05	3.500000e+04	5.000000
50%	2015.000000	4.500000e+05	6.000000e+04	5.000000
75%	2017.000000	6.750000e+05	9.800000e+04	5.000000
max	2020.000000	1.000000e+07	2.360457e+06	14.000000

```
In [7]: df.columns
```

```
Out[7]: Index(['name', 'year', 'selling_price', 'km_driven', 'fuel', 'seller_type',
              'transmission', 'owner', 'mileage', 'engine', 'max_power', 'torque',
              'seats'],
              dtype='str')
```

```
In [8]: df.shape
```

```
Out[8]: (8128, 13)
```

```
In [9]: len(df)
```

Out[9]: 8128

```
In [10]: df.dtypes
```

Out[10]: name str
year int64
selling_price int64
km_driven int64
fuel str
seller_type str
transmission str
owner str
mileage str
engine str
max_power str
torque str
seats float64
dtype: object

```
In [11]: df.info()
```

<class 'pandas.DataFrame'>
RangeIndex: 8128 entries, 0 to 8127
Data columns (total 13 columns):
Column Non-Null Count Dtype

0 name 8128 non-null str
1 year 8128 non-null int64
2 selling_price 8128 non-null int64
3 km_driven 8128 non-null int64
4 fuel 8128 non-null str
5 seller_type 8128 non-null str
6 transmission 8128 non-null str
7 owner 8128 non-null str
8 mileage 7907 non-null str
9 engine 7907 non-null str
10 max_power 7913 non-null str
11 torque 7906 non-null str
12 seats 7907 non-null float64
dtypes: float64(1), int64(3), str(9)
memory usage: 825.6 KB

```
In [12]: df.describe(include="all")
```

Out[12]:

	name	year	selling_price	km_driven	fuel	seller_type	transmission	owner	mileage	engine	max_power
count	8128	8128.000000	8.128000e+03	8.128000e+03	8128	8128	8128	8128	7907	7907	7913
unique	2058	NaN	NaN	NaN	4	3	2	5	393	121	322
top	Maruti Swift Dzire VDI	NaN	NaN	NaN	Diesel	Individual	Manual	First Owner	18.9 kmpl	1248 CC	74 bhp
freq	129	NaN	NaN	NaN	4402	6766	7078	5289	225	1017	377
mean	NaN	2013.804011	6.382718e+05	6.981951e+04	NaN	NaN	NaN	NaN	NaN	NaN	NaN
std	NaN	4.044249	8.062534e+05	5.655055e+04	NaN	NaN	NaN	NaN	NaN	NaN	NaN
min	NaN	1983.000000	2.999900e+04	1.000000e+00	NaN	NaN	NaN	NaN	NaN	NaN	NaN
25%	NaN	2011.000000	2.549990e+05	3.500000e+04	NaN	NaN	NaN	NaN	NaN	NaN	NaN
50%	NaN	2015.000000	4.500000e+05	6.000000e+04	NaN	NaN	NaN	NaN	NaN	NaN	NaN
75%	NaN	2017.000000	6.750000e+05	9.800000e+04	NaN	NaN	NaN	NaN	NaN	NaN	NaN
max	NaN	2020.000000	1.000000e+07	2.360457e+06	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
In [13]: df.owner
```

Out[13]: 0 First Owner
1 Second Owner
2 Third Owner
3 First Owner
4 First Owner
...
8123 First Owner
8124 Fourth & Above Owner
8125 First Owner
8126 First Owner
8127 First Owner
Name: owner, Length: 8128, dtype: str

```
In [14]: df ["owner"].unique()
```

Out[14]: <StringArray>
['First Owner', 'Second Owner', 'Third Owner',
 'Fourth & Above Owner', 'Test Drive Car']
Length: 5, dtype: str

```
In [15]: owner_map = {  
    "First Owner": 1,  
    "Second Owner": 2,  
    "Third Owner": 3,  
    "Fourth & Above Owner": 4,
```

```
        "Test Drive Car": 5
    }

df["owner"] = df["owner"].map(owner_map)
```

In [16]: df ["owner"].unique()

Out[16]: array([1, 2, 3, 4, 5])

In [17]: df.owner

Out[17]:

0	1
1	2
2	3
3	1
4	1
...	
8123	1
8124	4
8125	1
8126	1
8127	1

Name: owner, Length: 8128, dtype: int64

In [18]: df.fuel

Out[18]:

0	Diesel
1	Diesel
2	Petrol
3	Diesel
4	Petrol
...	
8123	Petrol
8124	Diesel
8125	Diesel
8126	Diesel
8127	Diesel

Name: fuel, Length: 8128, dtype: str

In [19]: df ["fuel"].unique()

Out[19]: <StringArray>
['Diesel', 'Petrol', 'LPG', 'CNG']
Length: 4, dtype: str

In [20]: df = df[~df["fuel"].isin(["LPG", "CNG"])]

In [21]: df.fuel.unique()

Out[21]: <StringArray>
['Diesel', 'Petrol']
Length: 2, dtype: str

In [22]: df.mileage

Out[22]:

0	23.4 kmpl
1	21.14 kmpl
2	17.7 kmpl
3	23.0 kmpl
4	16.1 kmpl
...	
8123	18.5 kmpl
8124	16.8 kmpl
8125	19.3 kmpl
8126	23.57 kmpl
8127	23.57 kmpl

Name: mileage, Length: 8033, dtype: str

In [23]: df["mileage"] = df["mileage"].str.split().str[0].astype(float)

In [24]: df ["mileage"].dtype

Out[24]: dtype('float64')

In [25]: df ["engine"].unique()

Out[25]: <StringArray>
['1248 CC', '1498 CC', '1497 CC', '1396 CC', '1298 CC', '1197 CC', '796 CC',
'1364 CC', '1399 CC', '1461 CC',
...
'2197 CC', '2999 CC', '1781 CC', '2650 CC', '1343 CC', '2446 CC', '3498 CC',
'2198 CC', '2776 CC', '1950 CC']
Length: 122, dtype: str

In [26]: df ["engine"]=df["engine"].str.split().str[0].astype(float)

In [27]: df ["engine"].dtype

Out[27]: dtype('float64')

In [28]: df ['max_power']

```
Out[28]: 0          74 bhp
         1      103.52 bhp
         2          78 bhp
         3          90 bhp
         4       88.2 bhp
         ...
        8123      82.85 bhp
        8124       110 bhp
        8125       73.9 bhp
        8126        70 bhp
        8127        70 bhp
        Name: max_power, Length: 8033, dtype: str

In [29]: df["max_power"]=df["max_power"].str.split().str[0].astype(float)

In [30]: df["name"]

Out[30]: 0          Maruti Swift Dzire VDI
         1      Skoda Rapid 1.5 TDI Ambition
         2      Honda City 2017-2020 EXi
         3      Hyundai i20 Sportz Diesel
         4      Maruti Swift VXI BSIII
         ...
        8123      Hyundai i20 Magna
        8124      Hyundai Verna CRDi SX
        8125      Maruti Swift Dzire ZDi
        8126      Tata Indigo CR4
        8127      Tata Indigo CR4
        Name: name, Length: 8033, dtype: str

In [31]: df["name"]=df["name"].str.split().str[0].astype(str)

In [32]: df["name"]

Out[32]: 0          Maruti
         1          Skoda
         2          Honda
         3          Hyundai
         4          Maruti
         ...
        8123      Hyundai
        8124      Hyundai
        8125      Maruti
        8126      Tata
        8127      Tata
        Name: name, Length: 8033, dtype: str

In [33]: df=df.drop(columns=["torque"])

In [34]: df.columns

Out[34]: Index(['name', 'year', 'selling_price', 'km_driven', 'fuel', 'seller_type',
               'transmission', 'owner', 'mileage', 'engine', 'max_power', 'seats'],
              dtype='str')

In [35]: df.seller_type.unique()

Out[35]: <StringArray>
['Individual', 'Dealer', 'Trustmark Dealer']
Length: 3, dtype: str

In [36]: df.engine.unique()

Out[36]: array([1248., 1498., 1497., 1396., 1298., 1197., 796., 1364., 1399.,
               1461., 993., nan, 1061., 1198., 1199., 998., 1591., 2179.,
               1368., 2982., 2494., 2143., 2477., 1462., 2755., 1968., 1798.,
               1196., 1373., 1598., 1998., 1086., 1194., 1172., 1405., 1582.,
               999., 2487., 1999., 3604., 2987., 1995., 1451., 1969., 2967.,
               2497., 1797., 1991., 2362., 1493., 1599., 1341., 1794., 799.,
               1193., 2696., 1495., 1186., 1047., 2498., 2956., 2523., 1120.,
               624., 1496., 1984., 2354., 814., 793., 1799., 936., 1956.,
               1997., 1499., 1948., 2997., 2489., 2499., 2609., 2953., 1150.,
               1994., 1388., 1527., 2199., 995., 2993., 1586., 1390., 909.,
               2393., 3198., 1339., 2835., 2092., 1595., 2496., 1596., 1597.,
               2596., 2148., 1299., 1590., 2231., 2694., 2200., 1795., 1896.,
               1796., 1422., 1489., 2359., 2197., 2999., 1781., 2650., 1343.,
               2446., 3498., 2198., 2776., 1950.])

In [37]: df.dtypes

Out[37]: name          str
         year        int64
         selling_price  int64
         km_driven     int64
         fuel          str
         seller_type    str
         transmission   str
         owner         int64
         mileage       float64
         engine        float64
         max_power     float64
         seats        float64
         dtype: object

In [38]: df.transmission.unique()
```

Out[38]: <StringArray>
['Manual', 'Automatic']
Length: 2, dtype: str

In [39]: df.name.unique()

Out[39]: <StringArray>
['Maruti', 'Skoda', 'Honda', 'Hyundai',
 'Toyota', 'Ford', 'Renault', 'Mahindra',
 'Tata', 'Chevrolet', 'Fiat', 'Datsun',
 'Jeep', 'Mercedes-Benz', 'Mitsubishi', 'Audi',
 'Volkswagen', 'BMW', 'Nissan', 'Lexus',
 'Jaguar', 'Land', 'MG', 'Volvo',
 'Daewoo', 'Kia', 'Force', 'Ambassador',
 'Ashok', 'Isuzu', 'Opel', 'Peugeot']
Length: 32, dtype: str

In [40]: df = df[df["owner"] !=5]

In [41]: df.owner.unique()

Out[41]: array([1, 2, 3, 4])

In [42]: df.selling_price

Out[42]: 0 450000
1 370000
2 158000
3 225000
4 130000
...
8123 320000
8124 135000
8125 382000
8126 290000
8127 290000
Name: selling_price, Length: 8028, dtype: int64

In [43]: y = np.log(df['selling_price'])

In [44]: y

Out[44]: 0 13.017003
1 12.821258
2 11.970350
3 12.323856
4 11.775290
...
8123 12.676076
8124 11.813030
8125 12.853176
8126 12.577636
8127 12.577636
Name: selling_price, Length: 8028, dtype: float64

In [45]: df.head()

Out[45]:

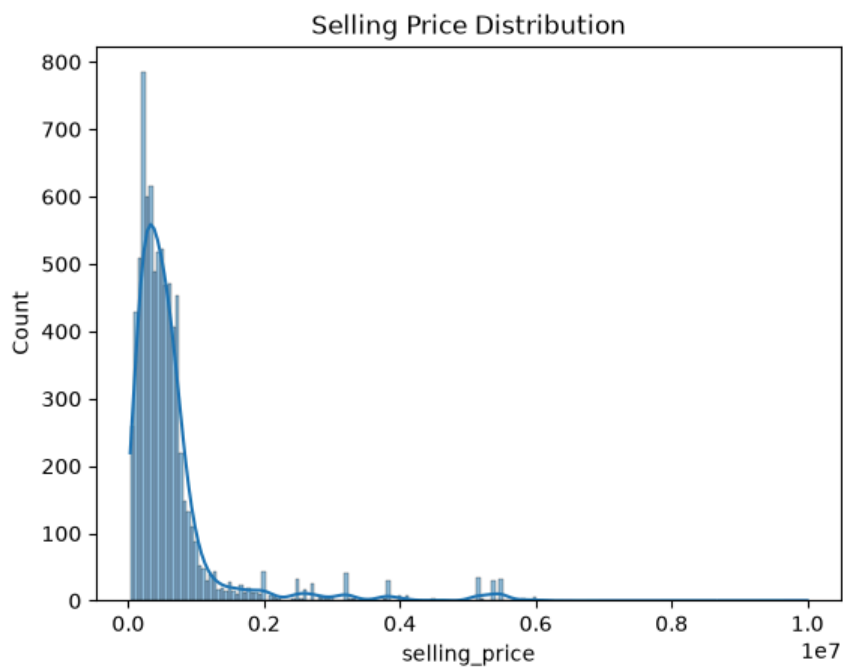
	name	year	selling_price	km_driven	fuel	seller_type	transmission	owner	mileage	engine	max_power	seats
0	Maruti	2014	450000	145500	Diesel	Individual	Manual	1	23.40	1248.0	74.00	5.0
1	Skoda	2014	370000	120000	Diesel	Individual	Manual	2	21.14	1498.0	103.52	5.0
2	Honda	2006	158000	140000	Petrol	Individual	Manual	3	17.70	1497.0	78.00	5.0
3	Hyundai	2010	225000	127000	Diesel	Individual	Manual	1	23.00	1396.0	90.00	5.0
4	Maruti	2007	130000	120000	Petrol	Individual	Manual	1	16.10	1298.0	88.20	5.0

In [46]: df.isnull().sum()

Out[46]: name 0
year 0
selling_price 0
km_driven 0
fuel 0
seller_type 0
transmission 0
owner 0
mileage 214
engine 214
max_power 208
seats 214
dtype: int64

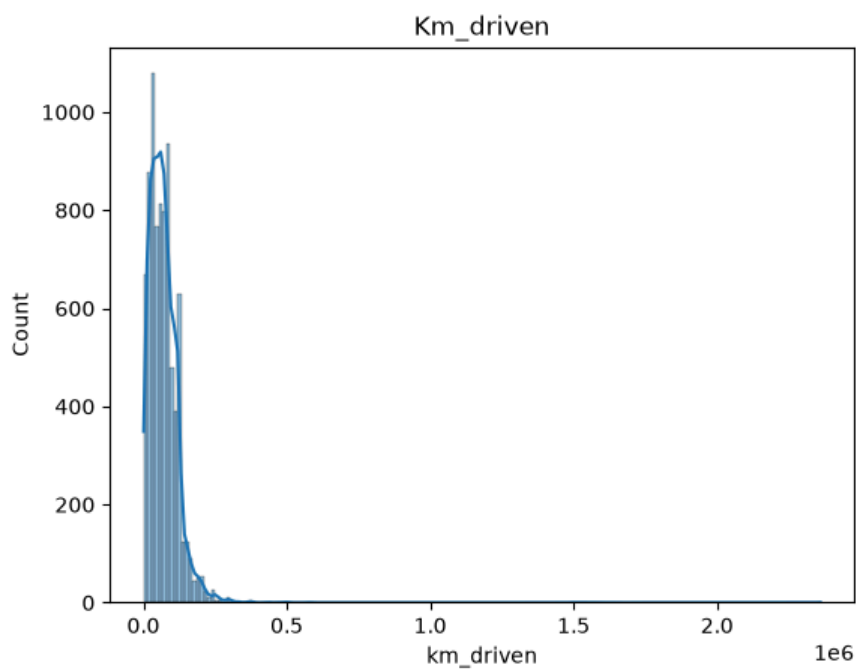
In [47]: import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(df["selling_price"], kde=True)
plt.title("Selling Price Distribution")
plt.show()



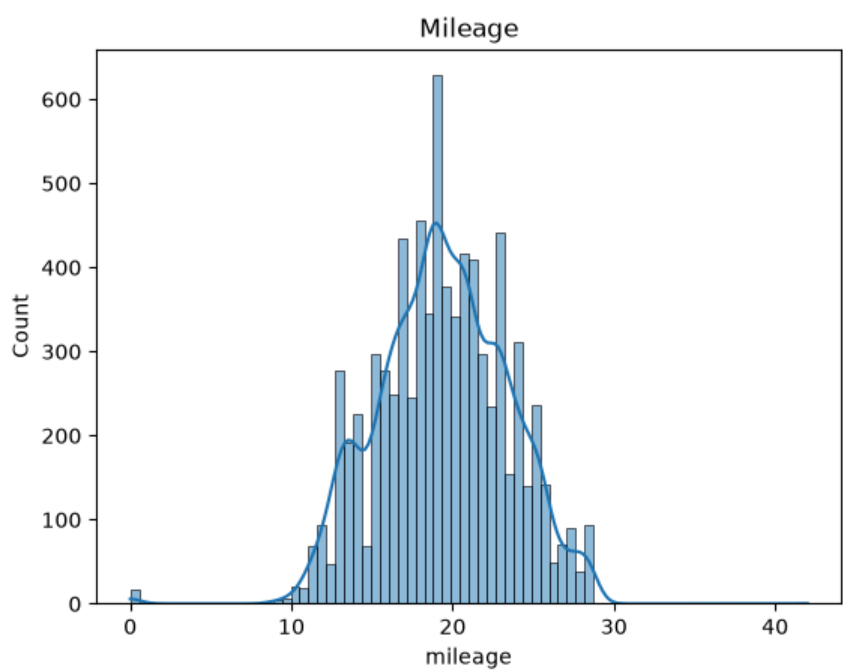
```
In [48]: import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(df["km_driven"], kde=True)
plt.title("Km_driven")
plt.show()
```



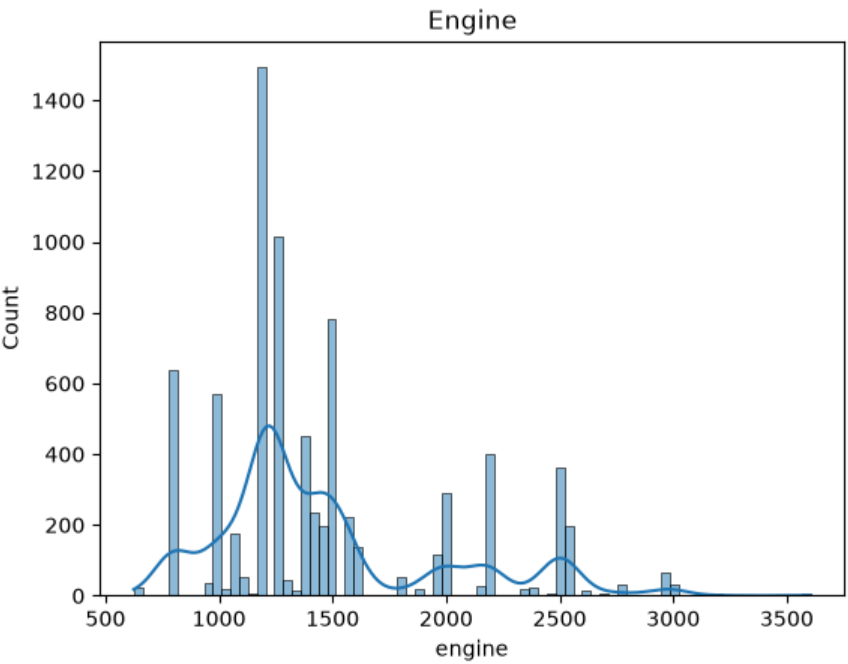
```
In [49]: import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(df["mileage"], kde=True)
plt.title("Mileage")
plt.show()
```



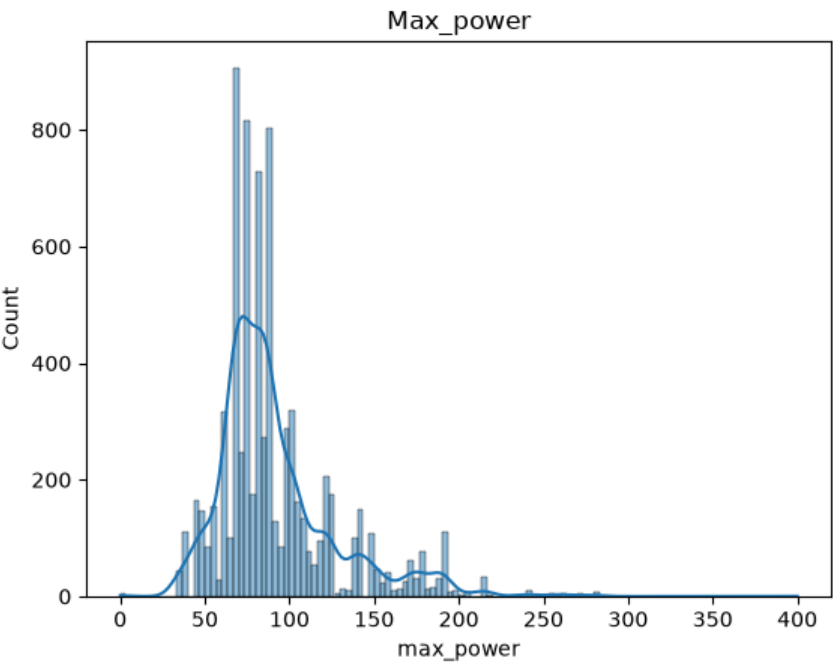
```
In [50]: import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(df["engine"], kde=True)
plt.title("Engine")
plt.show()
```



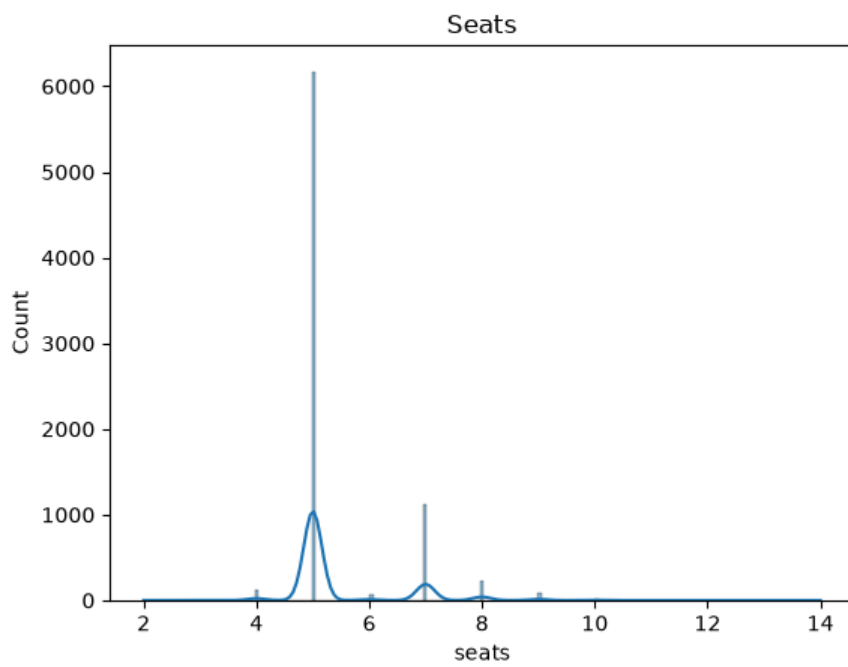
```
In [51]: import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(df["max_power"], kde=True)
plt.title("Max_power")
plt.show()
```



```
In [52]: import seaborn as sns
import matplotlib.pyplot as plt

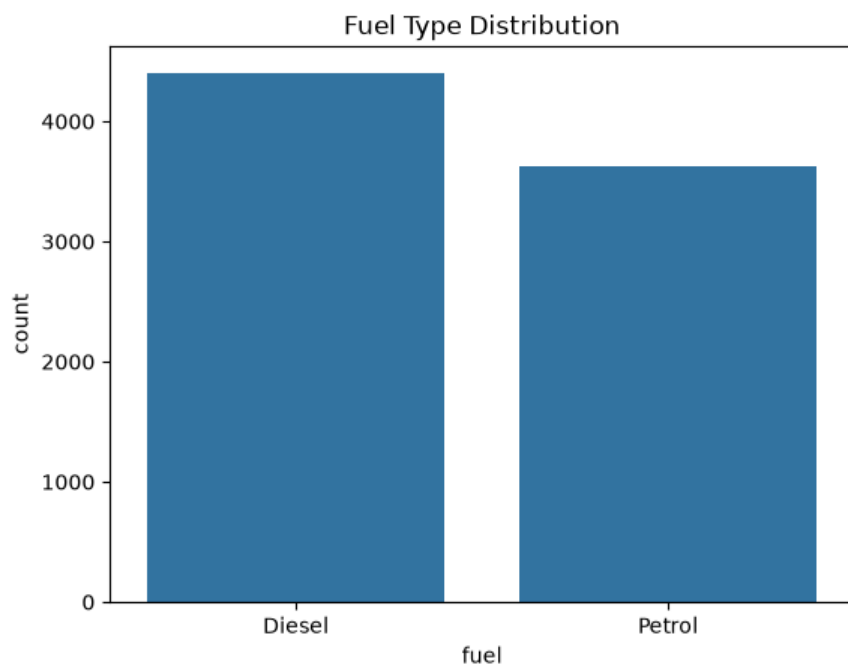
sns.histplot(df["seats"], kde=True)
plt.title("Seats")
plt.show()
```



```
In [53]: import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x=df["fuel"])

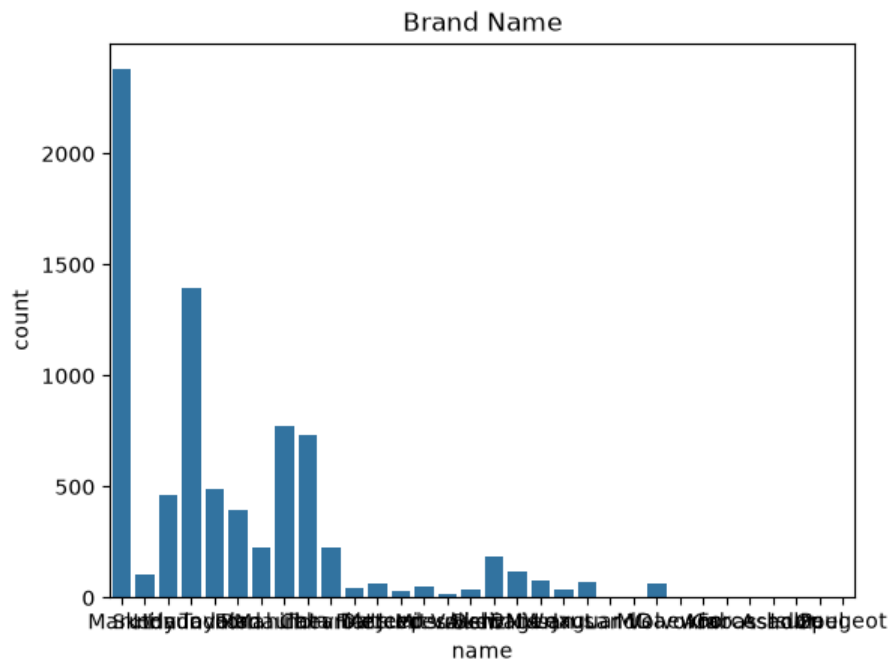
plt.title("Fuel Type Distribution")
plt.show()
```



```
In [54]: import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x=df["name"])

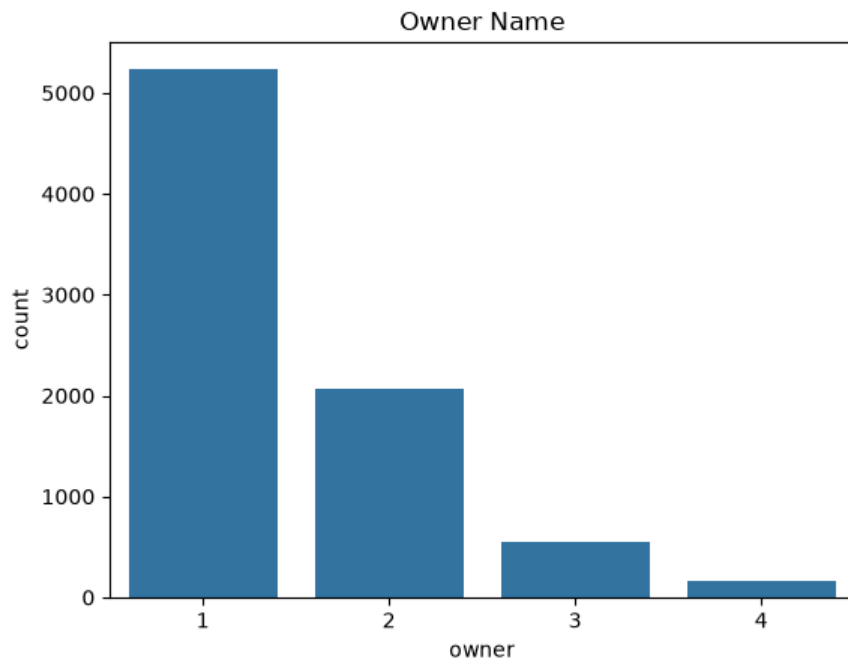
plt.title("Brand Name")
plt.show()
```

```
In [55]: import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x=df["owner"])

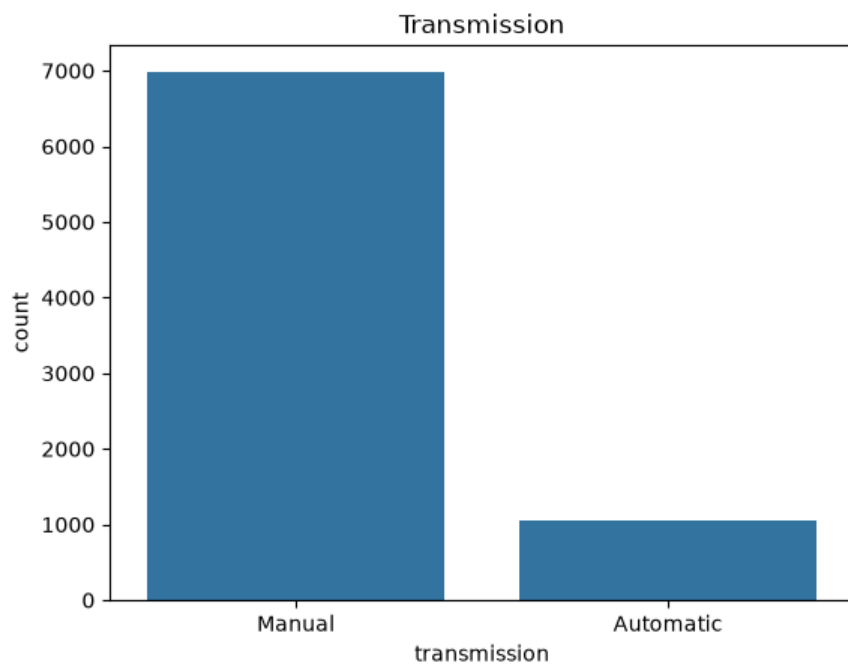
plt.title("Owner Name")
plt.show()
```



```
In [56]: import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x=df["transmission"])

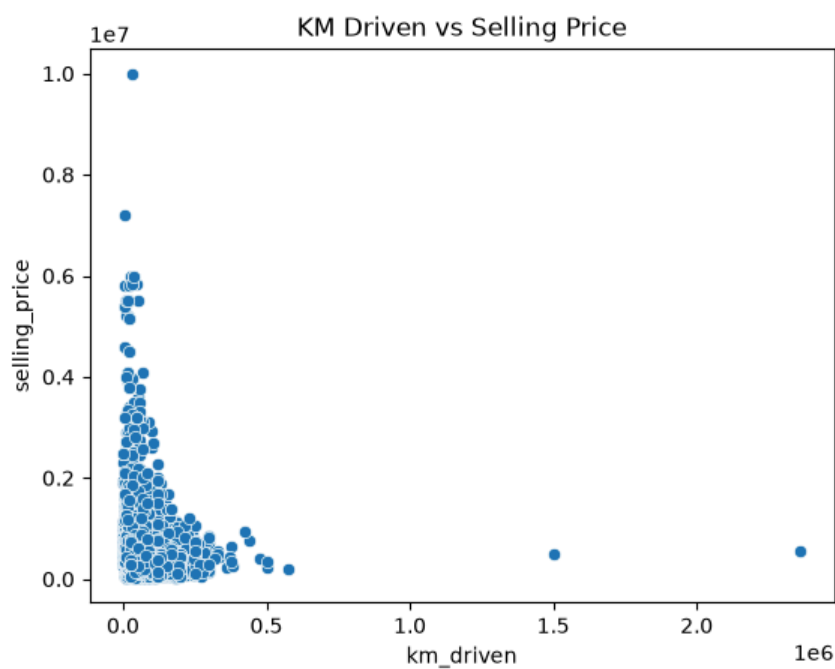
plt.title("Transmission")
plt.show()
```



```
In [57]: import seaborn as sns
import matplotlib.pyplot as plt

sns.scatterplot(
    data=df,
    x="km_driven",
    y="selling_price"
)

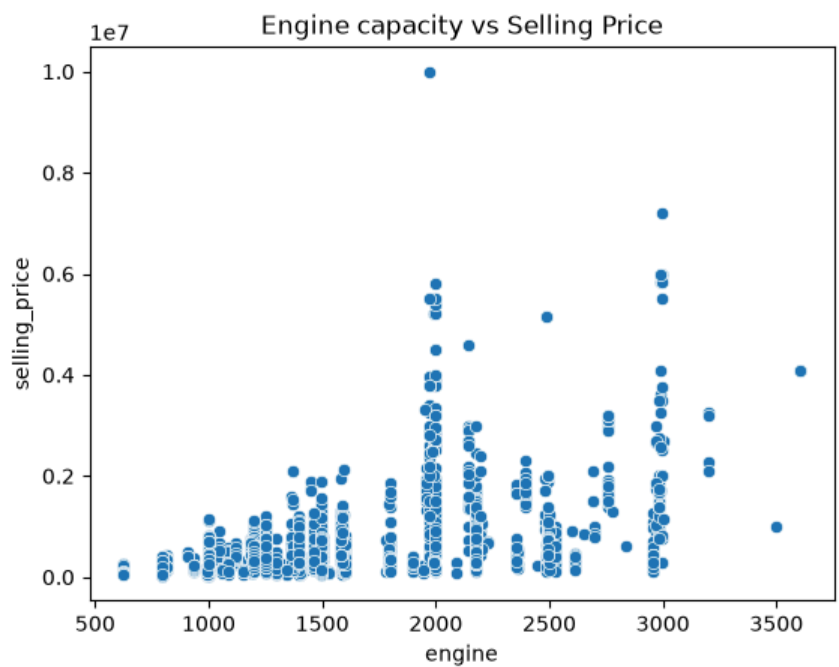
plt.title("KM Driven vs Selling Price")
plt.show()
```



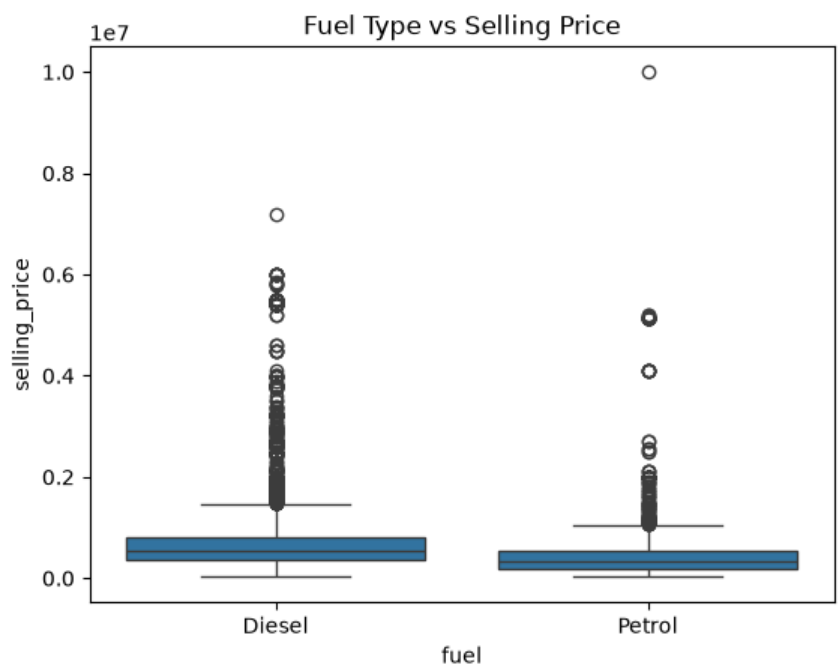
```
In [58]: import seaborn as sns
import matplotlib.pyplot as plt

sns.scatterplot(
    data=df,
    x="engine",
    y="selling_price"
)

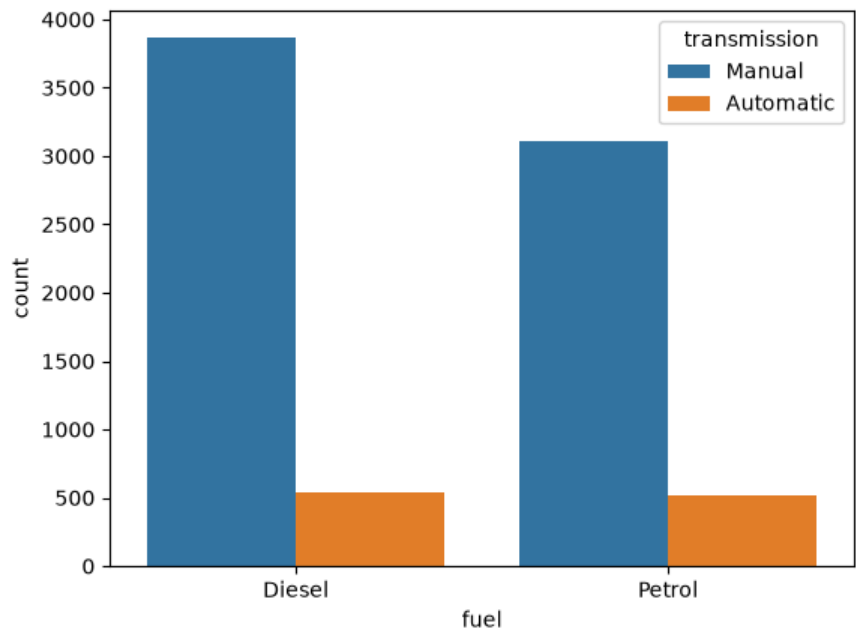
plt.title("Engine capacity vs Selling Price")
plt.show()
```



```
In [59]: sns.boxplot(  
    data=df,  
    x="fuel",  
    y="selling_price"  
)  
  
plt.title("Fuel Type vs Selling Price")  
plt.show()
```

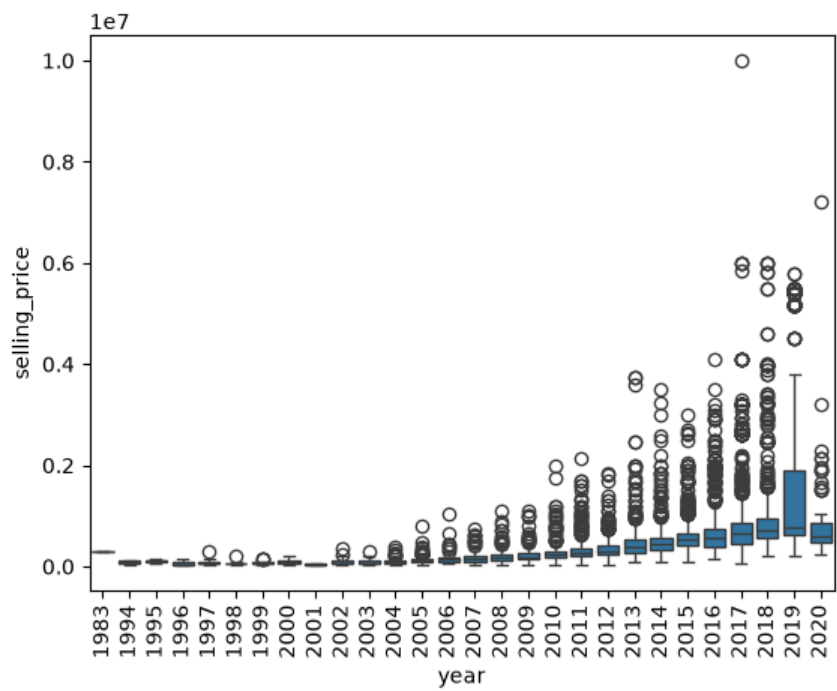


```
In [60]: sns.countplot(  
    data=df,  
    x="fuel",  
    hue="transmission"  
)  
  
plt.show()
```



```
In [61]: sns.boxplot(
    data=df,
    x="year",
    y="selling_price"
)

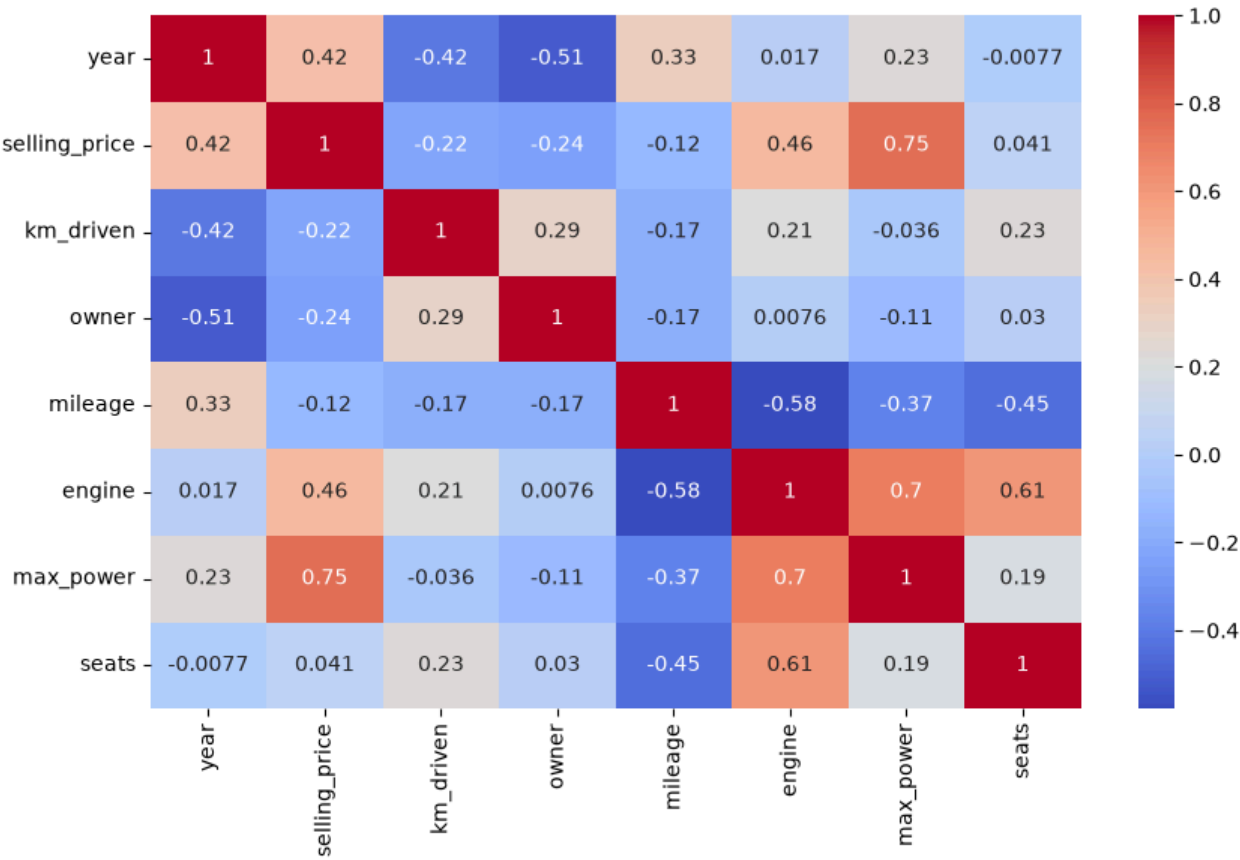
plt.xticks(rotation=90)
plt.show()
```



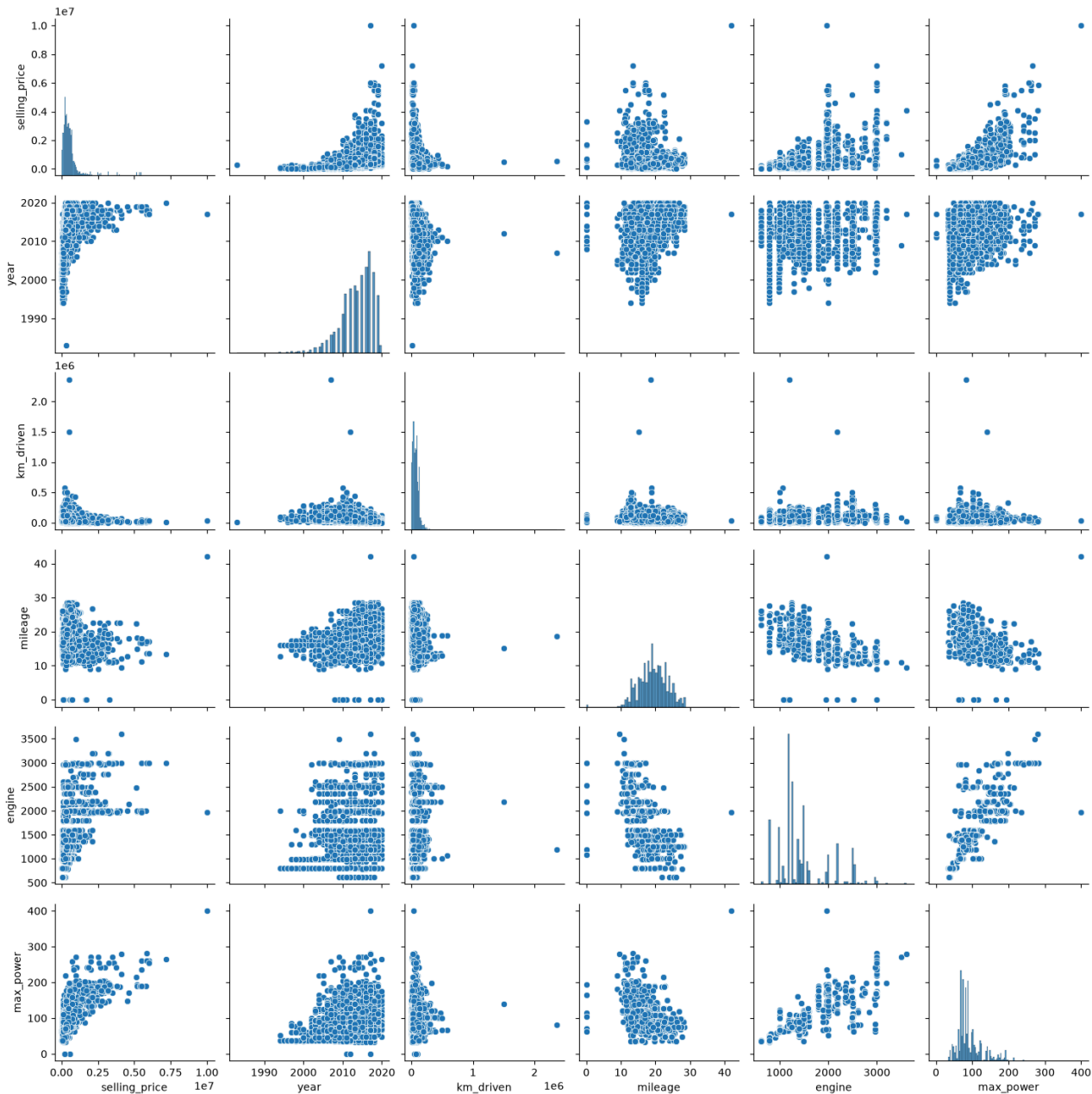
```
In [62]: plt.figure(figsize=(10,6))

sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap="coolwarm"
)

plt.show()
```



```
In [63]: sns.pairplot(
df[
    "selling_price",
    "year",
    "km_driven",
    "mileage",
    "engine",
    "max_power"
]
)
plt.show()
```



In [64]: `df.head()`

Out[64]:

	name	year	selling_price	km_driven	fuel	seller_type	transmission	owner	mileage	engine	max_power	seats
0	Maruti	2014	450000	145500	Diesel	Individual	Manual	1	23.40	1248.0	74.00	5.0
1	Skoda	2014	370000	120000	Diesel	Individual	Manual	2	21.14	1498.0	103.52	5.0
2	Honda	2006	158000	140000	Petrol	Individual	Manual	3	17.70	1497.0	78.00	5.0
3	Hyundai	2010	225000	127000	Diesel	Individual	Manual	1	23.00	1396.0	90.00	5.0
4	Maruti	2007	130000	120000	Petrol	Individual	Manual	1	16.10	1298.0	88.20	5.0

In [65]: `X = df.drop("selling_price", axis=1)`
`y = np.log(df['selling_price'])`

In [66]: `cat_cols = [`
 `"name",`
 `"fuel",`
 `"seller_type",`
 `"transmission"`
 `]`

 `num_cols = [`
 `"year",`
 `"km_driven",`
 `"owner",`
 `"mileage",`
 `"engine",`
 `"max_power",`
 `"seats"`
 `]`

In [67]: `X_encoded = pd.get_dummies(`
 `X,`
 `columns=cat_cols,`
 `drop_first=True`
 `)`

In [68]: `X_encoded.shape`

Out[68]: `(8028, 42)`

```
In [69]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_encoded,
    y,
    test_size=0.2,
    random_state=42
)
```

```
In [70]: X_train.head()
```

Out[70]:

	year	km_driven	owner	mileage	engine	max_power	seats	name_Ashok	name_Audi	name_BMW	...	name_Renault
2982	2016	59300	1	25.17	799.0	53.30	5.0	False	False	False	...	True
2430	2013	60000	4	22.74	796.0	47.30	5.0	False	False	False	...	False
6294	2009	56000	2	NaN	NaN	NaN	NaN	False	False	False	...	False
6255	2018	50000	1	17.60	2179.0	153.86	7.0	False	False	False	...	False
4253	1999	100000	3	14.80	1998.0	68.00	5.0	False	False	False	...	False

5 rows × 42 columns

```
In [71]: X_train.shape
```

```
Out[71]: (6422, 42)
```

```
In [72]: X_test.shape
```

```
Out[72]: (1606, 42)
```

```
In [73]: y_train.shape
```

```
Out[73]: (6422,)
```

```
In [74]: y_test.shape
```

```
Out[74]: (1606,)
```

```
In [75]: X_train.isnull().sum()
```

```
Out[75]: year                                0
km_driven                                0
owner                                    0
mileage                                171
engine                                171
max_power                             165
seats                                171
name_Ashok                             0
name_Audi                              0
name_BMW                              0
name_Chevrolet                         0
name_Daewoo                           0
name_Datsun                           0
name_Fiat                             0
name_Force                             0
name_Ford                             0
name_Honda                            0
name_Hyundai                          0
name_Isuzu                            0
name_Jaguar                           0
name_Jeep                             0
name_Kia                              0
name_Land                             0
name_Lexus                            0
name_MG                               0
name_Mahindra                         0
name_Maruti                           0
name_Mercedes-Benz                    0
name_Mitsubishi                       0
name_Nissan                           0
name_Opel                             0
name_Peugeot                          0
name_Renault                          0
name_Skoda                            0
name_Tata                             0
name_Toyota                           0
name_Volkswagen                       0
name_Volvo                            0
fuel_Petrol                           0
seller_type_Individual                 0
seller_type_Trustmark Dealer           0
transmission_Manual                    0
dtype: int64
```

```
In [76]: X_train["mileage"]=X_train["mileage"].fillna(
    X_train["mileage"].median()
)
```

```
In [77]: X_train["engine"]=X_train["engine"].fillna(
    X_train["engine"].mean()
)
```

```
)
```

```
In [78]: X_train["max_power"]=X_train["max_power"].fillna(  
        X_train["max_power"].median()  
        )
```

```
In [79]: X_train["seats"]=X_train["seats"].fillna(  
        X_train["seats"].median()  
        )
```

```
In [80]: X_test["mileage"]=X_test["mileage"].fillna(  
        X_train["mileage"].median()  
        )
```

```
In [81]: X_test["seats"]=X_test["seats"].fillna(  
        X_test["seats"].median()  
        )
```

```
In [82]: X_test["max_power"]=X_test["max_power"].fillna(  
        X_test["max_power"].median()  
        )
```

```
In [83]: X_test["engine"]=X_test["engine"].fillna(  
        X_test["engine"].mean()  
        )
```

```
In [84]: y.isnull().sum()
```

Out[84]: np.int64(0)

```
In [85]: from sklearn.preprocessing import StandardScaler
```

```
In [86]: scaler = StandardScaler()
```

```
In [87]: num_cols = [  
        "year",  
        "km_driven",  
        "owner",  
        "mileage",  
        "engine",  
        "max_power",  
        "seats"  
        ]
```

```
In [88]: X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
```

```
In [89]: X_test[num_cols] = scaler.transform(X_test[num_cols])
```

```
In [90]: X_train[num_cols].describe()
```

Out[90]:

	year	km_driven	owner	mileage	engine	max_power	seats
count	6.422000e+03	6.422000e+03	6.422000e+03	6.422000e+03	6.422000e+03	6.422000e+03	6.422000e+03
mean	-1.436022e-14	-5.753382e-17	-2.378802e-17	-3.319259e-17	8.630074e-17	-2.157518e-17	-6.306592e-17
std	1.000078e+00	1.000078e+00	1.000078e+00	1.000078e+00	1.000078e+00	1.000078e+00	1.000078e+00
min	-7.616115e+00	-1.178218e+00	-6.420159e-01	-4.884975e+00	-1.681454e+00	-2.604286e+00	-3.595133e+00
25%	-6.964462e-01	-5.979803e-01	-6.420159e-01	-6.505133e-01	-5.343207e-01	-6.416423e-01	-4.362954e-01
50%	2.920779e-01	-1.713351e-01	-6.420159e-01	-2.038507e-02	-4.322198e-01	-2.476914e-01	-4.362954e-01
75%	7.863400e-01	5.112971e-01	7.733724e-01	6.777970e-01	2.364410e-01	2.970132e-01	-4.362954e-01
max	1.527733e+00	3.908782e+01	3.604149e+00	2.273282e+00	4.284441e+00	5.416953e+00	4.828434e+00

```
In [91]: X_train.isnull().sum()
```



```
Out[91]: year                                0
km_driven                                0
owner                                    0
mileage                                0
engine                                  0
max_power                               0
seats                                    0
name_Ashok                             0
name_Audi                               0
name_BMW                                0
name_Chevrolet                          0
name_Daewoo                             0
name_Datsun                             0
name_Fiat                               0
name_Force                              0
name_Ford                               0
name_Honda                              0
name_Hyundai                            0
name_Isuzu                              0
name_Jaguar                             0
name_Jeep                               0
name_Kia                                0
name_Land                               0
name_Lexus                              0
name_MG                                 0
name_Mahindra                           0
name_Maruti                             0
name_Mercedes-Benz                      0
name_Mitsubishi                         0
name_Nissan                              0
name_Opel                               0
name_Peugeot                            0
name_Renault                            0
name_Skoda                              0
name_Tata                               0
name_Toyota                             0
name_Volkswagen                         0
name_Volvo                              0
fuel_Petrol                             0
seller_type_Individual                  0
seller_type_Trustmark Dealer            0
transmission_Manual                     0
dtype: int64
```

```
In [92]: from sklearn.linear_model import LinearRegression
```

```
In [93]: model = LinearRegression()
```

```
In [94]: model.fit(
    X_train,
    y_train
)
```

```
Out[94]: ▾ LinearRegression ⓘ ⓘ
          ▶ Parameters
          ▶ Fitted attributes
```

```
In [95]: y_pred=model.predict(X_test)
```

```
In [96]: print (y_pred)

[12.09477386 13.64555582 12.75676252 ... 10.94045126 13.34823585
12.80140198]
```

```
In [97]: model.coef_
```

```
Out[97]: array([ 4.62057772e-01, -1.96884541e-02, -4.73243775e-02, -2.85193031e-03,
 8.78001225e-02,  2.86803569e-01,  3.27595459e-02, -5.04095362e-01,
 6.86206324e-02,  3.22289871e-01, -6.20216653e-01,  7.07572797e-02,
-5.95153742e-01, -4.73297114e-01, -4.70406286e-01, -3.39324887e-01,
-1.68128840e-01, -2.45623184e-01, -1.99505745e-01,  2.14992402e-01,
-1.69619687e-01,  6.56366975e-02,  5.55086046e-01,  4.44956286e-01,
 1.18847297e-01, -3.43597804e-01, -1.99491861e-01,  2.39182145e-01,
-1.45713840e-02, -2.75218391e-01,  2.22044605e-16, -8.88178420e-16,
-3.30965269e-01, -2.92343785e-01, -6.69427129e-01, -1.81997107e-02,
-3.01362623e-01,  2.01251167e-01, -2.51660271e-01, -4.44530790e-02,
-2.23927135e-02, -7.48086736e-02])
```

```
In [98]: coef_df = pd.DataFrame({
    "Feature": X_train.columns,
    "Coefficient": model.coef_
})

coef_df
```

Out[98]:

	Feature	Coefficient
0	year	4.620578e-01
1	km_driven	-1.968845e-02
2	owner	-4.732438e-02
3	mileage	-2.851930e-03
4	engine	8.780012e-02
5	max_power	2.868036e-01
6	seats	3.275955e-02
7	name_Ashok	-5.040954e-01
8	name_Audi	6.862063e-02
9	name_BMW	3.222899e-01
10	name_Chevrolet	-6.202167e-01
11	name_Daewoo	7.075728e-02
12	name_Datsun	-5.951537e-01
13	name_Fiat	-4.732971e-01
14	name_Force	-4.704063e-01
15	name_Ford	-3.393249e-01
16	name_Honda	-1.681288e-01
17	name_Hyundai	-2.456232e-01
18	name_Isuzu	-1.995057e-01
19	name_Jaguar	2.149924e-01
20	name_Jeep	-1.696197e-01
21	name_Kia	6.563670e-02
22	name_Land	5.550860e-01
23	name_Lexus	4.449563e-01
24	name_MG	1.188473e-01
25	name_Mahindra	-3.435978e-01
26	name_Maruti	-1.994919e-01
27	name_Mercedes-Benz	2.391821e-01
28	name_Mitsubishi	-1.457138e-02
29	name_Nissan	-2.752184e-01
30	name_Opel	2.220446e-16
31	name_Peugeot	-8.881784e-16
32	name_Renault	-3.309653e-01
33	name_Skoda	-2.923438e-01
34	name_Tata	-6.694271e-01
35	name_Toyota	-1.819971e-02
36	name_Volkswagen	-3.013626e-01
37	name_Volvo	2.012512e-01
38	fuel_Petrol	-2.516603e-01
39	seller_type_Individual	-4.445308e-02
40	seller_type_Trustmark Dealer	-2.239271e-02
41	transmission_Manual	-7.480867e-02

In [99]: model.intercept_

Out[99]: np.float64(13.455474872074763)

In [100]: prediction_df = pd.DataFrame({
 "Actual Price": y_test,
 "Predicted Price": y_pred
})

prediction_df.head(10)

Out[100...

	Actual Price	Predicted Price
5948	12.323856	12.094774
6039	13.710150	13.645556
3069	12.676076	12.756763
6531	13.384728	13.176320
322	13.161584	13.402345
4648	12.043554	12.024359
3397	12.542545	12.970667
3635	13.122363	12.986358
3999	12.043554	12.434100
5633	12.721886	12.801987

In [101...

```
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(
    y_test,
    y_pred
)

print("MAE:", mae)
```

MAE: 0.1934624027764602

In [102...

```
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(
    y_test,
    y_pred
)

print("MSE:", mse)
```

MSE: 0.06440748427717784

In [103...

```
import numpy as np

rmse = np.sqrt(mse)

print("RMSE:", rmse)
```

RMSE: 0.2537862964724018

In [104...

```
from sklearn.metrics import r2_score

r2 = r2_score(
    y_test,
    y_pred
)

print("R² Score:", r2)
```

R² Score: 0.9095580100174817

In [105...

```
train_score = model.score(
    X_train,
    y_train
)

print("Training R²:", train_score)
```

Training R²: 0.9072145675342526

In [106...

```
test_score = model.score(
    X_test,
    y_test
)

print("Testing R²:", test_score)
```

Testing R²: 0.9095580100174817

In [107...

```
import pandas as pd
import numpy as np

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

# Predictions
y_pred = model.predict(X_test)

# Parameters
print("Intercept:")
print(model.intercept_)

print("\nCoefficients:")
```

```
coef_df = pd.DataFrame({
    "Feature": X_train.columns,
    "Coefficient": model.coef_
})

print(coef_df)

# Metrics
mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

print("\nModel Performance")
print("-----")
print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)

# Prediction comparison
results = pd.DataFrame({
    "Actual": y_test,
    "Predicted": y_pred
})

print("\nPrediction Results:")
print(results.head())
```

Intercept:
13.455474872074763

Coefficients:

	Feature	Coefficient
0	year	4.620578e-01
1	km_driven	-1.968845e-02
2	owner	-4.732438e-02
3	mileage	-2.851930e-03
4	engine	8.780012e-02
5	max_power	2.868036e-01
6	seats	3.275955e-02
7	name_Ashok	-5.040954e-01
8	name_Audi	6.862063e-02
9	name_BMW	3.222899e-01
10	name_Chevrolet	-6.202167e-01
11	name_Daewoo	7.075728e-02
12	name_Datsun	-5.951537e-01
13	name_Fiat	-4.732971e-01
14	name_Force	-4.704063e-01
15	name_Ford	-3.393249e-01
16	name_Honda	-1.681288e-01
17	name_Hyundai	-2.456232e-01
18	name_Isuzu	-1.995057e-01
19	name_Jaguar	2.149924e-01
20	name_Jeep	-1.696197e-01
21	name_Kia	6.563670e-02
22	name_Land	5.550860e-01
23	name_Lexus	4.449563e-01
24	name_MG	1.188473e-01
25	name_Mahindra	-3.435978e-01
26	name_Maruti	-1.994919e-01
27	name_Mercedes-Benz	2.391821e-01
28	name_Mitsubishi	-1.457138e-02
29	name_Nissan	-2.752184e-01
30	name_Opel	2.220446e-16
31	name_Peugeot	-8.881784e-16
32	name_Renault	-3.309653e-01
33	name_Skoda	-2.923438e-01
34	name_Tata	-6.694271e-01
35	name_Toyota	-1.819971e-02
36	name_Volkswagen	-3.013626e-01
37	name_Volvo	2.012512e-01
38	fuel_Petrol	-2.516603e-01
39	seller_type_Individual	-4.445308e-02
40	seller_type_Trustmark Dealer	-2.239271e-02
41	transmission_Manual	-7.480867e-02

Model Performance

MAE: 0.1934624027764602
MSE: 0.06440748427717784
RMSE: 0.2537862964724018
R2 Score: 0.9095580100174817

Prediction Results:

	Actual	Predicted
5948	12.323856	12.094774
6039	13.710150	13.645556
3069	12.676076	12.756763
6531	13.384728	13.176320
322	13.161584	13.402345

```
In [108... from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.linear_model import Ridge, Lasso
from xgboost import XGBRegressor

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
import pandas as pd
```

```
In [109... models = {

    "Random Forest": RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ),

    "XGBoost": XGBRegressor(
        n_estimators=100,
        learning_rate=0.1,
        random_state=42
    ),

    "Gradient Boosting": GradientBoostingRegressor(
        n_estimators=100,
        learning_rate=0.1,
        random_state=42
    ),

    "Ridge Regression": Ridge(
        alpha=1.0
    ),

    "Lasso Regression": Lasso(
        alpha=1.0
    )
}
```

```
In [110... results = []

for name, model in models.items():

    # Train
    model.fit(X_train, y_train)

    # Predict
    y_pred = model.predict(X_test)

    # Metrics
    mae = mean_absolute_error(y_test, y_pred)

    rmse = np.sqrt(
        mean_squared_error(y_test, y_pred)
    )

    r2 = r2_score(
        y_test,
        y_pred
    )

    results.append(
        {
            "Model": name,
            "MAE": mae,
            "RMSE": rmse,
            "R2 Score": r2
        }
    )
```

```
In [111... results_df = pd.DataFrame(results)

results_df.sort_values(
    by="R2 Score",
    ascending=False
)
```

Out[111...

	Model	MAE	RMSE	R2 Score
1	XGBoost	0.142895	0.198126	0.944879
0	Random Forest	0.138168	0.201823	0.942802
2	Gradient Boosting	0.161282	0.217585	0.933520
3	Ridge Regression	0.193556	0.253562	0.909718
4	Lasso Regression	0.629605	0.843901	-0.000038

```
In [112... xgb_model = models["XGBoost"]
```

```
In [113... importance = pd.DataFrame({
    "Feature": X_train.columns,
    "Importance": xgb_model.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)
```

```
)

importance
```

Out[113...

	Feature	Importance
5	max_power	0.321510
0	year	0.233911
4	engine	0.059521
34	name_Tata	0.047526
41	transmission_Manual	0.042083
38	fuel_Petrol	0.040782
35	name_Toyota	0.032508
19	name_Jaguar	0.024302
6	seats	0.022461
10	name_Chevrolet	0.019546
25	name_Mahindra	0.019002
26	name_Maruti	0.014329
20	name_Jeep	0.011054
12	name_Datsun	0.009505
36	name_Volkswagen	0.009215
9	name_BMW	0.009079
2	owner	0.008556
27	name_Mercedes-Benz	0.008521
13	name_Fiat	0.007386
3	mileage	0.006968
17	name_Hyundai	0.006102
15	name_Ford	0.005787
29	name_Nissan	0.005578
1	km_driven	0.005298
16	name_Honda	0.005186
22	name_Land	0.003831
33	name_Skoda	0.003424
32	name_Renault	0.003371
39	seller_type_Individual	0.003148
8	name_Audi	0.002391
14	name_Force	0.002150
37	name_Volvo	0.002125
28	name_Mitsubishi	0.001556
21	name_Kia	0.001343
40	seller_type_Trustmark Dealer	0.000945
7	name_Ashok	0.000000
18	name_Isuzu	0.000000
11	name_Daewoo	0.000000
24	name_MG	0.000000
23	name_Lexus	0.000000
30	name_Opel	0.000000
31	name_Peugeot	0.000000

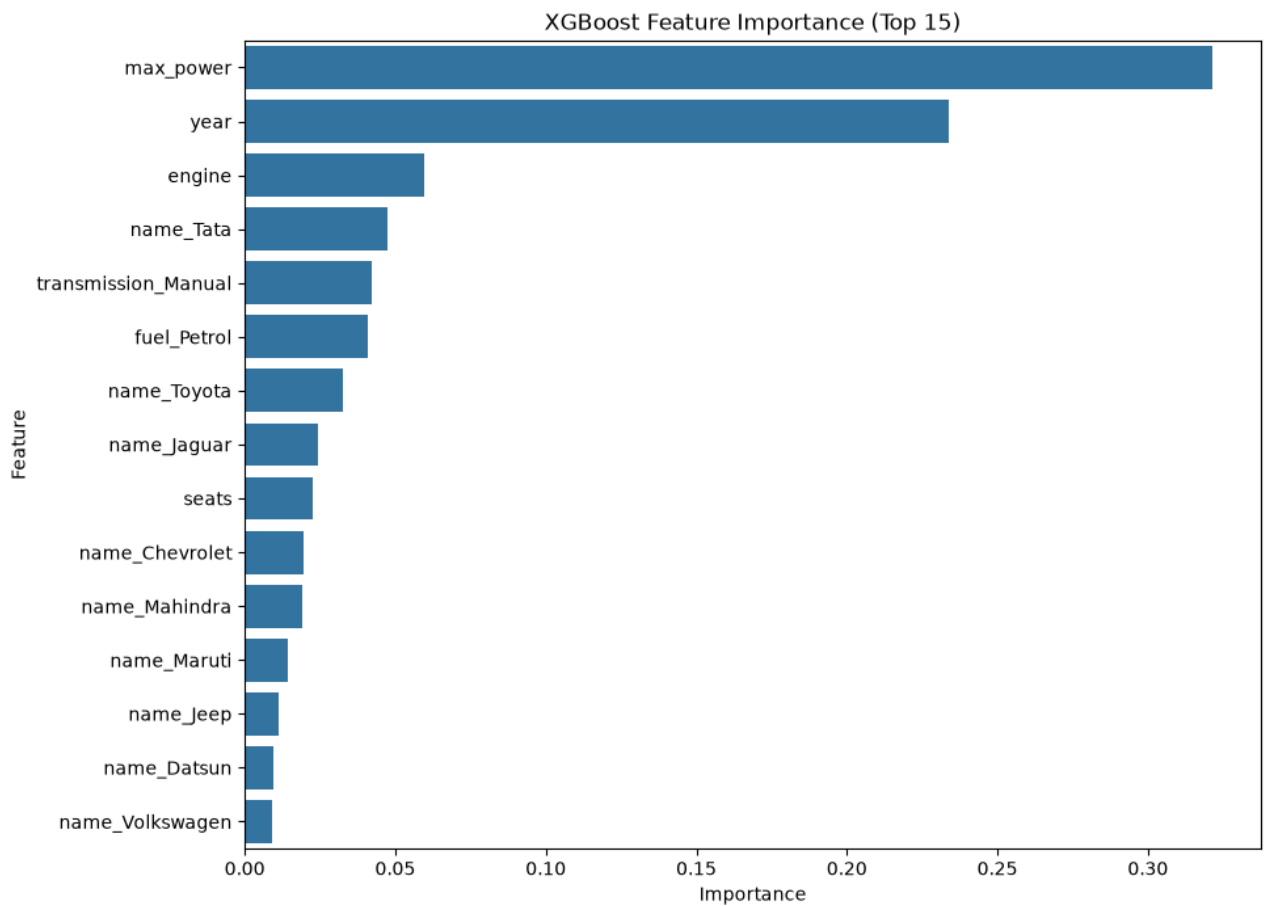
In [114...

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10,8))

sns.barplot(
    data=importance.head(15),
    x="Importance",
    y="Feature"
)

plt.title("XGBoost Feature Importance (Top 15)")
plt.show()
```



```
In [129... %pip install joblib

Requirement already satisfied: joblib in c:\users\user\downloads\ml assignment 1\venv\lib\site-packages (1.5.3)
Note: you may need to restart the kernel to use updated packages.
[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

In [116... import joblib

joblib.dump(
    xgb_model,
    "xgboost_car_price_model.pkl"
)

Out[116... ['xgboost_car_price_model.pkl']

In [127... joblib.dump(
    scaler,
    "scaler.pkl"
)

Out[127... ['scaler.pkl']

In [118... xgb_model = models["XGBoost"]

In [119... import joblib

joblib.dump(
    xgb_model,
    "xgb_car_price_model.pkl"
)

Out[119... ['xgb_car_price_model.pkl']

In [120... joblib.dump(
    X_train.columns,
    "model_columns.pkl"
)

Out[120... ['model_columns.pkl']

In [121... input_data = pd.DataFrame({
    "name": ["Maruti"],
    "year": [2014],
    "km_driven": [145500],
    "fuel": ["Diesel"],
    "seller_type": ["Individual"],
    "transmission": ["Manual"],
    "owner": [1],
    "mileage": [23.4],
    "engine": [1248],
    "max_power": [74],
    "seats": [5]
})

In [122... input_encoded = pd.get_dummies(
    input_data,
    columns=cat_cols,
```

```
        drop_first=True  
    )
```

```
In [123... model_columns = joblib.load(  
    "model_columns.pkl"  
)
```

```
In [124... input_encoded = input_encoded.reindex(  
    columns=model_columns,  
    fill_value=0  
)
```

```
In [125... prediction = model.predict(input_encoded)  
  
print(prediction[0])
```

12.978302823744217

```
In [126... import joblib  
  
joblib.dump(  
    scaler,  
    "scaler.pkl"  
)
```

Out[126... ['scaler.pkl']

In []: